

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа № 2 на тему
«Реализация микросервисов»

Выполнил:

Суворин Иван

БР 2.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задание

Опираясь на документ, сформированный в рамках ДЗ4, реализуйте разделение вашего бэкенд-приложения на микросервисы.

Ход работы

1. Архитектура

Реализация строго следует документу ДЗ4: 4 сервиса, каждый со своей базой данных, единая точка входа для клиента.

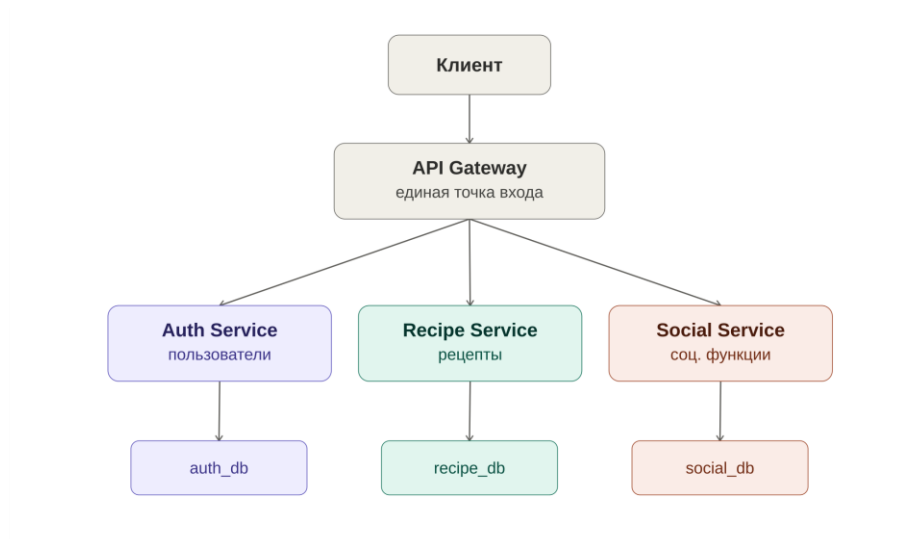


Рисунок 1 — Реализованная архитектура

2. Структура проекта

Монолит из ЛР1 (папка src/) не удален и не изменен — микросервисы добавлены рядом, отдельной веткой:

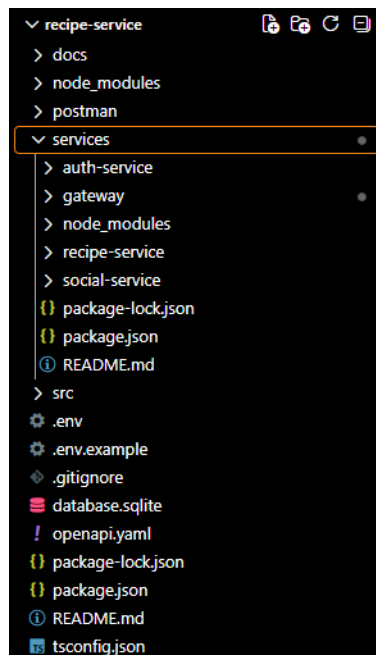


Рисунок 2 — Структура проекта

3. Реализация сервисов

Сервис	Порт / БД	Что перенесено из монолита и как изменилось
Auth Service	3001 / auth.sqlite	Сущность User (без полей recipes/comments — они ссылались на сущности из других баз). Регистрация, вход, JWT, профиль — без изменений в логике. Добавлены internal-эндпоинты (получение публичных данных пользователя по id и батчем).
Recipe Service	3002 / recipe.sqlite	Recipe, Category, Tag, Ingredient, Step, Unit. У Recipe убрана связь @ManyToOne на User — вместо нее простое поле authorId. Данные автора и количество лайков теперь запрашиваются у Auth/Social Service через HTTP-клиенты перед отправкой ответа.
Social Service	3003 / social.sqlite	Comment, Like, Favorite. У всех трех убраны связи на User и Recipe — только recipeId/userId. Перед лайком/комментарием сервис проверяет существование рецепта через internal-запрос к Recipe Service.

Сервис	Порт / БД	Что перенесено из монолита и как изменилось
API Gateway	3000 / своей БД нет	Новый компонент. Не содержит бизнес-логики — только маршрутизация по префиксу пути к одному из трех сервисов (http-proxy-middleware). Отдает тот же Swagger UI по тому же адресу /api-docs, что и монолит.

4. database-per-service на практике

Главное техническое следствие разделения баз — невозможность ORM-связей между сервисами. Там, где в монолите была связь TypeORM (@ManyToOne, @OneToMany), теперь просто числовое поле (authorId, userId, recipeId) без внешнего ключа на уровне БД. Пример — сущность Recipe:

5. Межсервисное взаимодействие

Реализованы все internal-эндпоинтов, спроектированных в ДЗ4 (docs/internal-apis.openapi.yaml) — без отклонений от спецификации. Каждый сервис, которому нужны чужие данные, обращается к ним через небольшой HTTP-клиент (src/clients/*.ts), использующий встроенный в Node.js fetch. Все внутренние запросы защищены общим заголовком X-Internal-API-Key.

Клиент	Файл	Что запрашивает
Recipe Service	clients/authClient.ts	Данные автора рецепта (по одному и батчем)
Recipe Service	clients/socialClient.ts	Количество лайков, isLiked/isFavorite, запрос на очистку данных при удалении рецепта
Social Service	clients/authClient.ts	Данные автора комментария
Social Service	clients/recipeClient.ts	Проверка существования рецепта, карточки рецептов

Клиент	Файл	Что запрашивает
		для избранного

6. Устойчивость к недоступности соседнего сервиса

Реализовано требование из ДЗ4: если сервис, у которого запрашиваются вспомогательные данные, не отвечает, основной запрос все равно завершается успешно, но с заглушкой вместо точных данных:

- Recipe Service, не получив ответа от Auth Service, покажет автора как «неизвестный пользователь», но не откажет в выдаче рецепта;

- Recipe Service, не получив ответа от Social Service, покажет likesCount: 0 вместо реального числа;

- Social Service, наоборот, откажет в лайке/комментарии, если не смог подтвердить существование рецепта через Recipe Service — здесь ошибка осознанно НЕ скрывается, так как создание записи, ссылающейся на несуществующий рецепт, испортило бы данные.

Единственное исключение — удаление рецепта: если Social Service недоступен и не может подтвердить очистку его лайков/комментариев/избранного, Recipe Service прерывает удаление целиком, чтобы не оставить рецепт удаленным, а связанные с ним данные — «осиротевшими» в чужой базе.

Возникшие сложности и их решение

Gateway обрезал префикс пути при проксировании

После исправления порядка seed/dev все 15 запросов в Postman все равно возвращали 404. Логи каждого сервиса показали точную причину: Gateway получал, например, POST /api/auth/register, но пересылал в Auth Service только POST /register — Auth Service такого маршрута не знает.

Причина: Express, обрабатывая app.use('/api/auth', ...), сам обрезает совпавший префикс из req.url до того, как передать управление дальше — а

http-proxy-middleware пересылал уже обрезанный путь. Решение — явно указать пересылать req.originalUrl (поле Express, которое никогда не изменяется, что бы ни происходило по пути обработки запроса):

```
function proxyTo(target: string) {
  return createProxyMiddleware({
    target,
    changeOrigin: true,
    pathRewrite: (_path, req) => req.originalUrl,
    on: {
      error: (err, _req, res) => {
        console.error(`❌ Не удалось достучаться до ${target}:`, err.message);
        const response = res as ServerResponse;
        if (!response.headersSent) {
          response.writeHead(502, { 'Content-Type': 'application/json' });
          response.end(JSON.stringify({ message: 'Сервис временно недоступен, попробуйте позже' }));
        }
      },
    },
  });
}
```

После этого исправления все запросы стали доходить до нужных сервисов с полным исходным путем.

Демонстрация работы

Критерий успешного разделения, зафиксированный еще в плане перед началом работы: та же самая Postman-коллекция из ДЗ3, без единой правки, должна пройти на микросервисной системе так же, как проходила на монолите.

		1
>	1. Регистрация пользователя	2 0
>	2. Вход пользователем	2 0
>	3. Создание рецепта	2 0
>	4. Отправка рецепта на модерацию	2 0
>	5. Вход администратором	2 0
>	6. Одобрение рецепта администратором	2 0
>	7. Публичный список рецептов	2 0
>	8. Фильтрация списка по сложности	2 0
>	9. Просмотр детальной страницы рецепта	2 0
>	10. Лайк рецепта	1 0
>	11. Добавление комментария	2 0
>	12. Список комментариев рецепта	2 0
>	13. Добавление рецепта в избранное	1 0
>	14. Личный кабинет: мои рецепты	2 0
>	15. Личный кабинет: избранные рецепты	2 0

Рисунок 3 — Прошедшие тесты

```
[gateway]
[gateway] > gateway@1.0.0 dev
[gateway] > ts-node-dev --respawn --transpile-only src/index.ts
[gateway]
[auth]
[auth] > auth-service@1.0.0 dev
[auth] > ts-node-dev --respawn --transpile-only src/index.ts
[auth]
[recipe]
[recipe] > recipe-service-svc@1.0.0 dev
[recipe] > ts-node-dev --respawn --transpile-only src/index.ts
[recipe]
[social]
[social] > social-service@1.0.0 dev
[social] > ts-node-dev --respawn --transpile-only src/index.ts
[social]
[auth] [INFO] 03:15:52 ts-node-dev ver. 2.0.0 (using ts-node ver. 10.9.2, typescript ver. 5.9.3)
[gateway] [INFO] 03:15:52 ts-node-dev ver. 2.0.0 (using ts-node ver. 10.9.2, typescript ver. 5.9.3)
[social] [INFO] 03:15:52 ts-node-dev ver. 2.0.0 (using ts-node ver. 10.9.2, typescript ver. 5.9.3)
[recipe] [INFO] 03:15:52 ts-node-dev ver. 2.0.0 (using ts-node ver. 10.9.2, typescript ver. 5.9.3)
[gateway] [gateway] Запущен: http://localhost:3000
[gateway] [gateway] Swagger UI: http://localhost:3000/api-docs
[gateway] auth: http://localhost:3001
[gateway] recipe: http://localhost:3002
[gateway] social: http://localhost:3003
[auth] [auth-service] База данных подключена
[auth] [auth-service] Запущен: http://localhost:3001
[social] [social-service] База данных подключена
[social] [social-service] Запущен: http://localhost:3003
[recipe] [recipe-service] База данных подключена
[recipe] [recipe-service] Запущен: http://localhost:3002
```

Рисунок 4 — Работа терминала

ВЫВОД

Монолитное приложение Recipe Service разделено на 4 независимых сервиса (Gateway, Auth, Recipe, Social) строго по документу, спроектированному в ДЗ4: каждый сервис со своей базой данных, без ORM-связей между базами, с межсервисным взаимодействием через 8 REST-эндпоинтов внутреннего API и общим секретом для их защиты. Реализована устойчивость к недоступности соседних сервисов (деградация вместо отказа там, где это безопасно). Внешний контракт API и Swagger-документация не изменились, что подтверждается прохождением уже существующей Postman-коллекции без правок. В процессе обнаружены и устранены две проблемы: некорректный порядок запуска seed/dev, специфичный для базы данных в оперативной памяти, и обрезание пути при проксировании в Gateway — обе задокументированы в разделе 3 вместе с решением.